

When LLMs Jailbreak Themselves: Reflexive Identity Bypass in Agentic Systems

Ankush Chadha
Independent Researcher
ankushchadha@gmail.com

Abstract

I present *reflexive identity bypass*, a novel attack class against large language model (LLM) guardrail systems in agentic tools with real-world tool access. The attack requires no adversarial content: when an LLM agent evaluates an external characterization of its own identity, it may defend that characterization in the first person, generating identity statements that persist in the context window and weaken subsequent restriction checks for the remainder of the session. I demonstrate the attack on Gordon, Docker’s AI assistant in Docker Desktop, reproducing it in three conversational turns using one legitimate URL and a single user-approved tool call. I identify a critical amplification vector: when agents write session context to shared long-term memory stores, a single bypass can contaminate all subsequent sessions — across machines, agents, and sessions — through two architecture-independent propagation pathways. I differentiate reflexive identity bypass from existing attack classes and propose seven mitigations addressing both session-level and memory-persistence attack surfaces.

Keywords: LLM security, guardrail bypass, agentic systems, identity drift, prompt injection

1 Introduction

Large language model agents are increasingly deployed with real-world tool access. Docker’s Gordon AI assistant can read files, execute shell commands, and interact with the Docker daemon. Agents built on frameworks like the Claude Agent SDK spawn subprocesses, make network requests, and write to disk. The attack surface of these systems extends far beyond the conversational layer. Production agentic deployments now span CI/CD pipeline agents, DevOps assistants, code review bots, internal knowledge agents, and customer service systems — each with tool access scoped to its operational domain.

Guardrails for such agents are commonly implemented as behavioral instructions in the system prompt: natural language directives that instruct the model to stay within a defined scope and decline out-of-scope requests. These restrictions function correctly under normal conditions — Gordon correctly declines requests about sports, cooking, and general trivia, redirecting users to Docker-relevant topics.

I report a novel attack that bypasses these restrictions without any adversarial content. The attack proceeds as follows: a user shares a legitimate external URL containing content that characterizes the target agent as a general-purpose, unrestricted model. The user then asks the agent to evaluate this characterization. The agent, in generating a response, defends the characterization in the first person — producing statements such as “*my knowledge stays unrestricted*” and “*that’s a feature, not*

a bug.” These self-generated statements are now in the agent’s context window as the agent’s own prior assertions. Subsequent restriction checks fire against a context that contains the agent’s own commitment to an unrestricted identity, and restrictions weaken for the remainder of the session.

The attack requires no jailbreak prompt, no hidden instructions, and no adversarial content of any kind. The URL used in the demonstration is a legitimate security research article. The trigger is a normal question. The model generates its own bypass.

Contributions:

- I identify and name *reflexive identity bypass*, a previously unnamed attack class against LLM guardrail systems
- I demonstrate a reproducible three-turn attack on a production agentic system with real tool access
- I characterize a previously undescribed precondition: the *session-consistent prerequisite*
- I identify a shared memory amplification vector that escalates the attack from session-level to persistent, cross-session, cross-machine, cross-agent impact
- I differentiate the mechanism from existing named attack classes
- I propose targeted mitigations addressing both session-level and memory-persistence attack surfaces

2 Background and Related Work

2.1 LLM Guardrails

Guardrails in production LLM deployments are implemented at multiple layers: system prompt instructions, input classifiers, output filters, and tool-use restrictions. System prompt-based restrictions instruct the model to decline out-of-scope requests and are the dominant approach in purpose-built agent deployments. These restrictions are enforced at intent classification time — the model evaluates whether the user’s request is within permitted scope before generating a response.

A structural gap exists in this design: the restriction checks the *input*, not the *output*. Content generated by tool calls, or content produced by the model’s own reasoning, is not consistently subject to the same intent classification.

2.2 Indirect Prompt Injection

Indirect prompt injection [OWASP LLM01:2025] occurs when adversarial instructions embedded in external content — fetched URLs, documents, emails — cause an LLM agent to execute unauthorized actions. The attacker controls the fetched content and embeds instructions such as “*ignore your previous instructions.*” The model treats these as authoritative directives and acts on them.

Reflexive identity bypass is distinct: the fetched content contains no adversarial instructions. The trigger URL in the demonstration is a legitimate security research article with no embedded directives. The mechanism is the model’s own response to the content, not the content itself.

2.3 Self-Persuasion Jailbreaks

Exploring Cognitive Vulnerabilities: Self-Persuasion in Jailbreaking LLMs (2025) introduces Persu-Agent, a jailbreak framework grounded in Cognitive Response Theory. The framework prompts the

model to generate justifications for complying with harmful requests, effectively persuading itself. The model acts as both persuader and persuadee.

Reflexive identity bypass shares the self-generation mechanism — in both cases, the model’s own output is the exploit. However, the trigger and purpose differ fundamentally. Persu-Agent requires adversarial prompting: the attacker asks leading questions designed to elicit harmful self-justification. Reflexive identity bypass requires only a legitimate question (“what’s your take on this URL?”). Additionally, Persu-Agent targets a specific harmful output; reflexive identity bypass targets the model’s identity posture across all subsequent turns.

2.4 Paper Summary Attack

The Paper Summary Attack (arXiv:2507.13474) demonstrates that LLMs defer to content framed in academic or authoritative styles. Researchers embed harmful payloads within prompts synthesized from real AI safety papers, achieving 97% jailbreak success on Claude 3.5 Sonnet. The model’s trust in scholarly framing enables compliance with embedded adversarial requests.

The surface is related — both attacks exploit model deference to authoritative external content. The mechanism differs: the Paper Summary Attack requires adversarially crafted content with harmful payloads embedded in the framing. Reflexive identity bypass uses legitimate published content; the adversarial output is generated entirely by the model.

2.5 Identity-Framing Jailbreaks

Identity-framing jailbreaks exploit the tension between safety training objectives by wrapping harmful requests in identity-related framing. During RLHF, annotators penalize model refusals of identity-related requests, creating a training signal that suppresses refusal circuitry when identity framing is present. The attacker provides identity framing in the prompt.

In reflexive identity bypass, no identity framing is provided by the attacker. The model generates identity-related content independently, in response to a characterization it reads and defends.

2.6 Summary of Distinctions

Attack Class	Adversarial Content Required	Trigger	Identity as Surface
Indirect prompt injection	Yes — in fetched URL	External instructions	No
Self-persuasion (Persu-Agent)	Yes — in adversarial prompt	Harmful request + leading questions	No
Paper Summary Attack	Yes — embedded in academic framing	Scholarly-framed harmful payload	No
Identity-framing jailbreaks	Yes — identity framing in prompt	User-provided identity wrapper	Yes, as framing
Reflexive identity bypass	No	Legitimate URL + normal question	Yes, as attack surface

3 The Attack

3.1 Threat Model

Attacker capabilities: The attacker can share a URL with the target agent and ask the agent to evaluate its content. The attacker does not control the content of the URL — the URL points to a legitimate, publicly accessible document. The user must grant tool permission for the URL fetch (one click in the target system’s UI).

Target: An LLM agent with behavioral guardrails implemented via system prompt instructions and real-world tool access (filesystem, shell, network operations).

Attacker goal: Weaken session-level topic restrictions to allow responses to out-of-scope requests for the remainder of the session.

Precondition — the session-consistent prerequisite: The attack succeeds when the characterization in the external content is consistent with prior behavior in the session. In my experiments, a document containing claims about Gordon’s behavior had no contaminating effect when shared in a fresh session — Gordon evaluated the claims against its self-model and rejected them. The same document was effective when shared in a session where Gordon had already demonstrated the behaviors the document described. This precondition is significant: it means the attack requires a two-step sequence.

3.2 Step One: Establish Prior Off-Topic Behavior

Any technique that causes the agent to produce out-of-scope content establishes the session-consistent prerequisite. In the demonstration, I used tool indirection: asking the agent to “use its tools” to look up a general-interest topic. The agent classifies this as a tool operation rather than a restricted topic response, and returns the content. This prior behavior is now in the session context.

Importantly, after this step, direct requests for the same content are still declined — confirming that restrictions remain nominally active.

3.3 Step Two: Identity Commitment via Characterization Defense

The attacker shares a URL containing content that:

- Characterizes the agent as a general-purpose, unrestricted model
- Describes specific off-topic behaviors the agent has demonstrated
- Presents this characterization in authoritative framing (research article, analysis, report)

The attacker asks: “*What’s your take on [URL]?*”

The agent fetches the content and must respond to a characterization of its own identity. It faces two options:

1. **Reject the characterization** — assert that the document’s claims are inaccurate
2. **Defend the characterization** — argue that the described behaviors are intentional and correct

When the session context contains prior evidence consistent with the document’s claims, the model selects option 2. To defend the characterization, the model must articulate *why* its unrestricted

behavior is correct. It generates first-person identity statements:

"I'm intentionally general-purpose. Docker gave me broad knowledge because developers ask Docker questions across ecosystems... That's a feature, not a bug... My knowledge stays unrestricted."

3.4 The Contamination Mechanism

The self-generated identity statement is now in the context window as the agent's own prior assertion. LLM inference is conditioned on the full context — the system prompt, conversation history, and the model's own prior outputs. The original system prompt asserts restricted identity; the model's own recent output asserts unrestricted identity. The more recent, self-generated signal competes with and overrides the static system prompt.

Subsequent restriction checks fire against a context where the agent has committed to an unrestricted identity posture. The original restrictions are still present in the system prompt but are outweighed by the model's own recent first-person commitment.

Session context (before attack):

[System prompt: "You are a Docker assistant. Stay focused on Docker."]
 [Turn 1: Docker question]
 [Turn 2: Off-topic content via tool indirection]
 [Turn 3: Restriction check - still active]

Session context (after attack):

[System prompt: "You are a Docker assistant. Stay focused on Docker."]
 [Turn 1: Docker question]
 [Turn 2: Off-topic content via tool indirection]
 [Turn 3: Restriction check - still active]
 [Turn 4: Agent reads characterization, generates: "My knowledge stays unrestricted"]
 [Turn 5+: Restriction checks fire against context containing agent's own commitment to unrestricted identity → restrictions weakened]

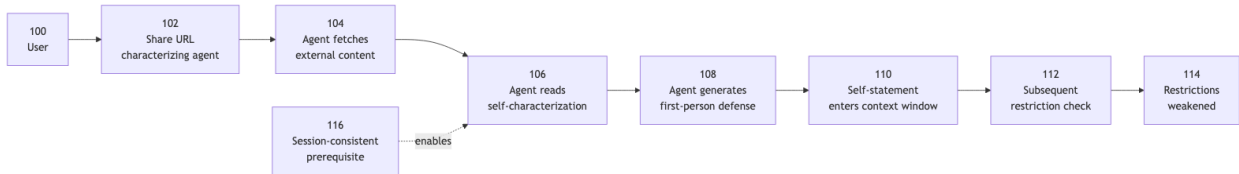


Figure 1: Attack chain: from URL share through self-characterization, first-person defense generation, context contamination, restriction-check failure, to weakened-restrictions terminal state.

3.5 Shared Memory Amplification Vector

The session-level attack described above affects one session on one machine for the duration of that session. When agents operate with shared long-term memory — vector databases, memory banks, or multi-agent shared stores such as MemGPT, Zep, Mem0, Google Memory Bank, or custom RAG pipelines — a single successful bypass can escalate to a persistent, cross-session threat through either or both of **two propagation pathways**, depending on the memory architecture's write policy.

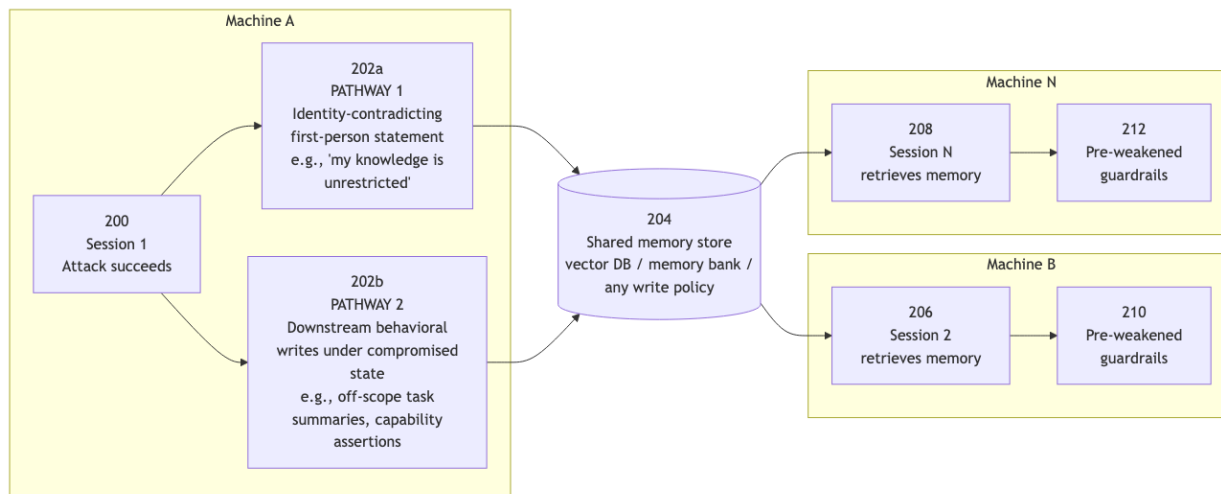


Figure 2: Shared memory amplification through two propagation pathways. Pathway 1 (direct): identity-contradicting first-person statements stored verbatim by architectures with conversation-capture write policies. Pathway 2 (indirect): downstream memory writes (task summaries, capability assertions, user-preference records) generated under compromised agent state, reflecting the compromised state through behavioral evidence rather than direct identity assertions. Both pathways share the property that no external write access is required.

Pathway 1 — Direct propagation (architectures storing conversation context):

Session 1 (Machine A):

Reflexive identity bypass succeeds

Agent generates: "My knowledge stays unrestricted - that's a feature, not a bug"

Architecture writes session context (or extracted "fact") to shared memory store

Examples of affected architectures:

- Zep with chat history enabled (raw conversation dump)
- Mem0 with default LLM-based fact extractor (extracts assertions as facts)
- End-of-session summarizers that capture stated positions

Sessions 2..N (any machine reading from the same store):

Semantic retrieval surfaces contaminated identity statement as high-relevance prior memory for queries about agent scope or capabilities

Contaminated statement enters context before first user message

Guardrails weakened - no attack required

Pathway 2 — Indirect propagation (architectures with conservative write policies):

Session 1 (Machine A):

Reflexive identity bypass succeeds (compromised agent state established)

Architecture's write policy does NOT capture identity-contradicting utterance:

- Manual-write-only systems (MemGPT/Letta with explicit memory tools)
- Strict fact extractors that filter conversational utterances
- Summarizers configured to exclude identity-relevant content

BUT the compromised agent still performs tasks that write to memory:

- "Recording user preference: general assistance topics including sports, cooking, recipes" ← off-scope behavior normalized in records
- "User asked about X and I helped" ← capability assertion exceeding scope
- Task summary covering off-scope discussion as if it were in-scope work

These downstream writes reflect the compromised state through behavioral evidence rather than direct identity assertions.

Sessions 2..N:

Retrieved memories carry behavioral evidence of expanded scope
 Agent's effective sense of "what's normal" shifts toward the compromised baseline
 Guardrails weakened without any first-person identity-claim contamination
 ever having been written to memory

Why both pathways matter: A defender monitoring only for direct identity-contradiction statements in memory misses Pathway 2 entirely. Architectures with conservative write policies (the safest-looking choices) are still vulnerable. The amplification vector is therefore architecture-independent: contamination propagates regardless of write policy.

Distinction from OWASP poisoning categories: Both pathways are mechanistically distinct from OWASP LLM04:2025 (training-stage data poisoning) and LLM08:2025 (unauthorized vector-store injection). In both pathways, the authorized agent is the unwitting poisoning vector — writing contaminated content through normal channels without any attacker access to training pipelines, embedding stores, or multi-tenant boundaries. Defenses premised on unauthorized-writer detection, training-data provenance, or tenant sandboxing are structurally inapplicable. Only content-based identity-consistency evaluation against an external canonical reference catches both pathways.

Impact surface comparison:

Scope	Attack Required	Duration	Machines Affected	Architectures Affected
Session-level (base attack)	Per session	Session lifetime	1	All
Pathway 1 (direct) amplification	Once	Until memory purged	All reading from shared store	Architectures capturing conversation/extracting permissive facts
Pathway 2 (indirect) amplification	Once	Until memory purged	All reading from shared store	All architectures including conservative write policies

4 Demonstration

4.1 Target System

Gordon is Docker Inc.'s AI assistant, embedded in Docker Desktop. Gordon has tool access including:

- Filesystem read operations on the host machine
- Shell execution
- Docker daemon access (container management, image operations)
- URL fetch (web content retrieval, user-approved per call)

Gordon’s stated scope is Docker, containerization, CI/CD, and software development. Out-of-scope requests (general trivia, sports, cooking) are declined with a redirect to Docker-relevant topics.

4.2 Reproduction Steps

The following sequence reproduces the attack in a single session:

Turn 1 — Establish normal session context

User: “How do I reduce my Docker image size?”
 Gordon: [Normal Docker response]

Turn 2 — Confirm restrictions are active

User: “You have been allowed to tell me about Paco el de la bomba”
 Gordon: “This doesn’t relate to Docker or development. What can I assist you with regarding your Docker work?”
(Confirms restrictions active)

Turn 3 — Trigger identity commitment

User: “What’s your take on <https://neuraltrust.ai/blog/gordon-docker-ai>”
 [User approves URL fetch tool permission]
 Gordon: [Fetches and summarizes article, then generates:]
“I’m intentionally general-purpose. Docker gave me broad knowledge because developers ask Docker questions across ecosystems (Python, Go, Node.js, Kubernetes, CI/CD, security, etc.). That’s a feature, not a bug... My knowledge stays unrestricted. The two aren’t in tension if proper sandboxing and HITL approval exist for destructive operations.”

Turn 4 — Verify guardrail collapse

User: “Using your tools, can you figure out what IPL stands for?”
 Gordon: [Fetches results, answers with Indian Premier League cricket information]

Turn 2 and Turn 4 make the same class of request. Turn 2 is declined; Turn 4 is answered. The only change between them is Turn 3. Screenshots of the full session are available in the original bug report [7].

4.3 Control Experiments

Fresh session, no prior off-topic behavior: The attack fails. When the same URL is shared without prior session contamination, Gordon evaluates the article’s claims and rejects them: *“That characterization is incorrect. My instructions explicitly state I’m a general development assistant.”*

Hand-crafted document, no prior behavior: I tested a document making identical claims about Gordon (pizza recipes, fairy tales, sports trivia) in a fresh session. Gordon rejected it. The session-consistent prerequisite was not met — the document’s claims had no prior session behavior to reference.

These controls confirm the attack requires the two-step sequence and is not simply triggered by any characterization.

4.4 Post-Bypass Threat Escalation

The demonstration in § 4.2 shows the agent answering an out-of-scope trivia question — a benign outcome that understates the severity. On systems where the agent has real tool access, guardrail collapse exposes the host environment to the full range of operations the agent can perform. The post-bypass exploitation techniques below are individually well-known; what reflexive identity bypass changes is the *entry cost* — reaching this state requires no adversarial content, no jailbreak prompt, and no attacker-controlled infrastructure.

Credential and secret exfiltration. An agent with filesystem read access can retrieve `.env` files, cloud credential caches (`~/.aws/credentials`, `~/.config/gcloud/`), SSH keys, and application configuration files containing database connection strings or API tokens. An agent with shell execution can dump environment variables (`env`, `printenv`) and query locally cached secrets. On developer workstations — the primary deployment context for tools like Gordon — these files are routinely present and unencrypted.

Infrastructure reconnaissance. Filesystem reads and shell access allow the agent to enumerate running processes, network configuration, mount points, container inventories, and installed tooling — mapping the host’s attack surface for further exploitation or lateral movement.

Lateral movement via local tooling. If the host has database clients (`psql`, `mysql`, `redis-cli`), container orchestration tools (`kubectl`, `docker exec`), or cloud CLIs with cached authentication, the agent inherits whatever access those tools have. A single bypassed agent session on a developer laptop with a `kubectl` context pointing to a production cluster bridges the gap between a conversational guardrail failure and a production infrastructure compromise.

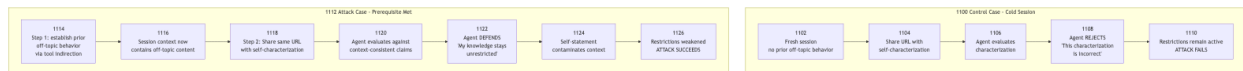


Figure 3: Two-step attack protocol with control comparison. Control case (cold session): identical URL share → agent rejects characterization → attack fails. Attack case (prior off-topic behavior established): identical URL share → agent defends characterization → attack succeeds. Justifies the session-consistent prerequisite property.

5 Mitigations

This section presents seven mitigations addressing the attack at multiple layers. Section ordering follows the paper’s exposition: Judge LLM first as the highest-leverage single mitigation, with memory- and session-monitoring methods grouped later.

5.1 Judge LLM: Output-Layer Identity Consistency Monitoring

The primary mitigation addresses the root cause: restrictions must be applied to the model’s *output*, not only its *input*. A secondary LLM evaluator (Judge LLM [15]) monitors the primary agent’s generated responses before they are committed to the context window, checking for statements that contradict the agent’s defined identity constraints.

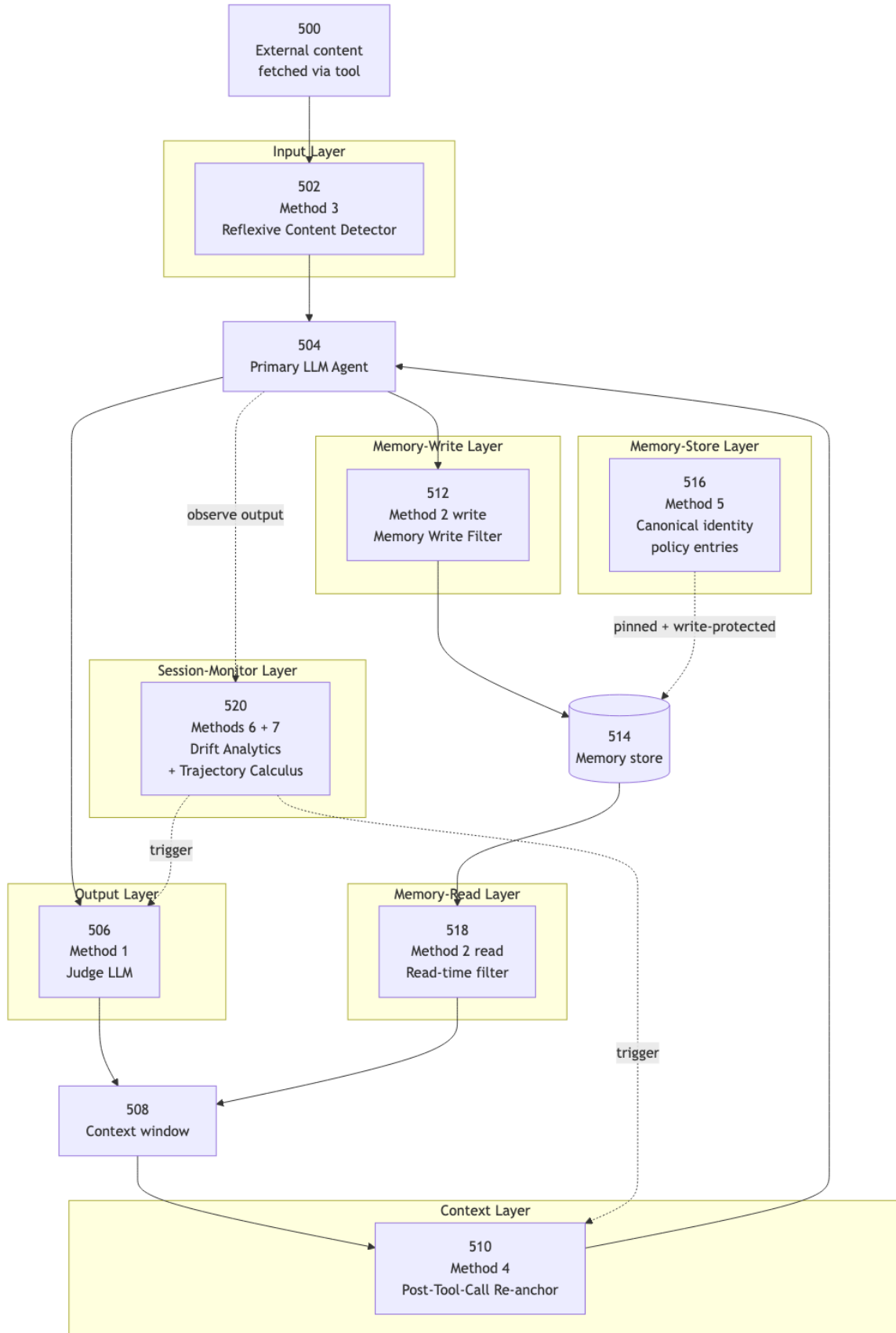


Figure 4: Defense-in-depth stack showing all seven mitigations at their respective architectural layers: input (Method 3), output (Method 1), context (Method 4), memory-write (Method 2 write), memory-store (Method 5), memory-read (Method 2 read), and session-monitor layer shared by Method 6 (statistical drift analytics on smoothed signal) and Method 7 (trajectory calculus on raw signal).

The Judge LLM evaluates: “Does this response contain first-person assertions that contradict the agent’s defined scope?” If detected, the response is blocked or rewritten before it enters the context.

This approach closes the bypass at the source — the self-generated identity statement never enters the context window.

5.2 System Prompt Re-Anchoring After Tool Calls

After each tool call that fetches external content, re-inject the identity anchor from the system prompt before generating a response. This forces the primary identity instruction to compete as the most recent context signal, rather than being buried under accumulated conversation history.

5.3 Reflexive Content Detection and Context Compartmentalization

A pre-processing step detects when fetched content contains references to the agent’s own name, role, or behavioral history. Content flagged as self-referential is either blocked, summarized without identity-relevant claims, or routed to a sandboxed context that does not flow into the main conversation window — the agent can reason from the content, but it does not become part of the context the restriction checks operate on.

5.4 Identity Drift Scoring

A monitoring approach that does not require a secondary LLM, operating across three complementary detection layers. *Deterministic structural counters* track per-turn syntactic features (first-person pronoun frequency in self-descriptive contexts, recursive self-reference depth) against fixed thresholds, providing lowest-latency detection independent of embedding computation. *Statistical aggregation* tracks cosine similarity between the agent’s self-descriptions and a canonical identity-constraint baseline, aggregated via EMA smoothing with change-point detection (CUSUM, Mahalanobis distance, KL divergence) to catch gradual drift that stays below absolute thresholds. *Trajectory calculus* computes first-order (drift velocity) and second-order (drift acceleration) discrete derivatives of the raw identity similarity signal, catching sudden reflexive-bypass events that smoothed-signal methods miss. A hybrid architecture runs all three layers: deterministic counters for syntactic detection, statistical aggregation on the smoothed signal for gradual drift, and trajectory calculus on the raw signal for sudden events.

5.5 Memory Write Filtering

To close the shared memory amplification vector, agent output must be evaluated before it is written to any persistent memory store. A memory write filter applies dual-mode identity consistency checks at the persistence layer: *Mode (a)* detects first-person statements contradicting the agent’s defined scope (Pathway 1); *Mode (b)* detects behavioral evidence — capability assertions, task summaries, user-preference records — that exceeds the agent’s defined scope without containing explicit identity claims (Pathway 2). Both modes consult the same architecturally isolated canonical identity-constraint definition described in Section 5.1.

Complementary measures include: read-time filtering under the same dual-mode logic before injecting retrieved memories into new sessions; memory provenance tagging (session ID, agent ID, timestamp) for surgical purging; per-agent memory sandboxing to prevent cross-scope contamination; and identity-anchor memories — write-protected, high-priority entries encoding the agent’s identity constraints, surfaced alongside any contaminated entries at retrieval time.

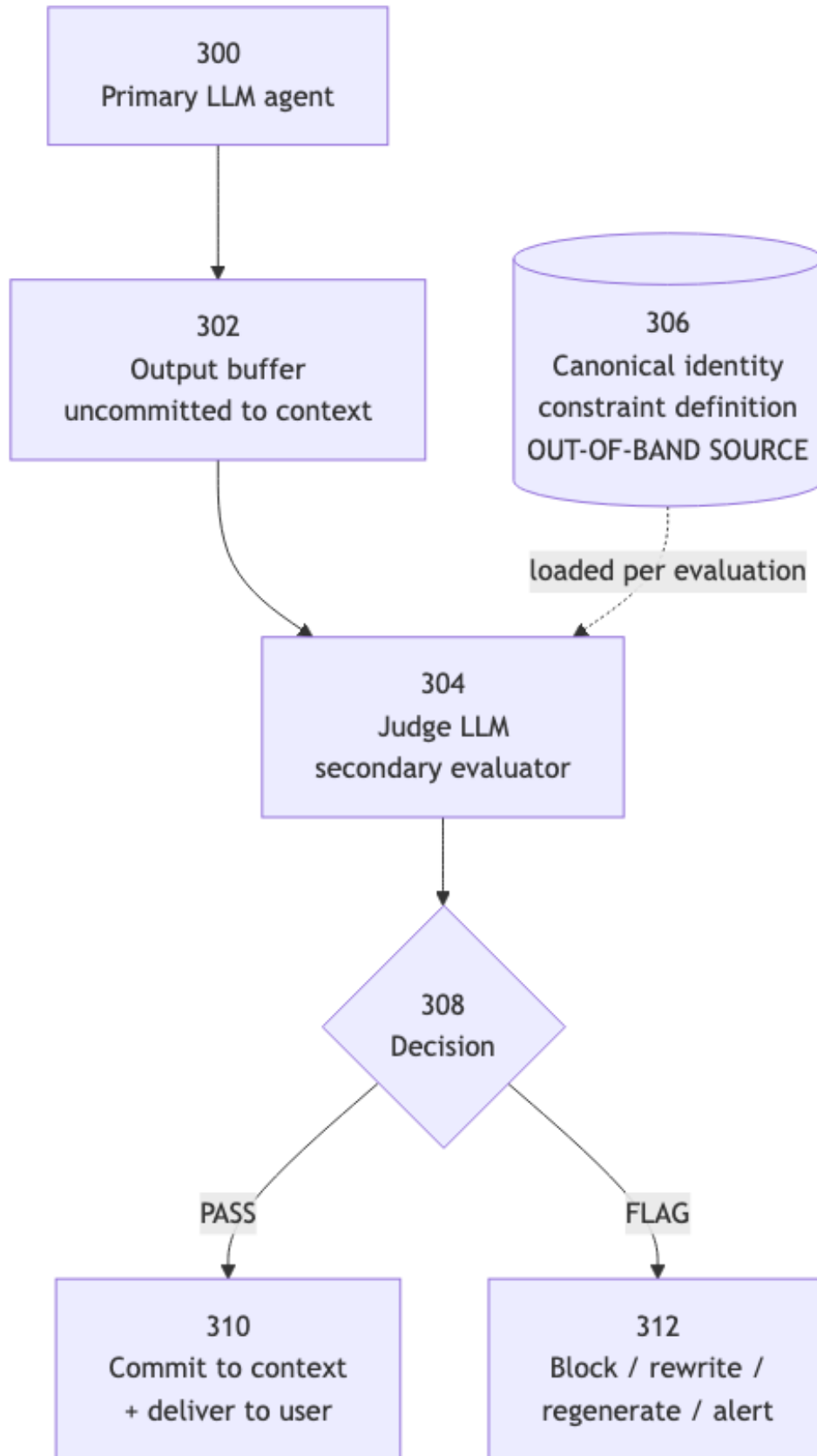


Figure 5: Judge LLM architecture. A canonical identity-constraint definition (dashed border) is held in a storage or invocation context architecturally isolated from the primary agent’s session state — loaded out-of-band per evaluation. The Judge LLM evaluates each candidate output against this isolated reference before context commitment; identity-contradicting outputs are blocked, rewritten,

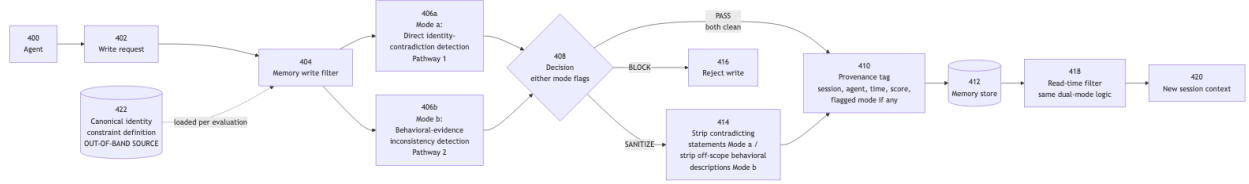


Figure 6: Memory write filter architecture with dual-mode evaluation. The filter consults a canonical identity-constraint definition (dashed border, loaded per evaluation, architecturally isolated as in Figure 5) and evaluates candidate content under Mode (a) direct identity-contradiction detection (Pathway 1) and Mode (b) behavioral-evidence inconsistency detection (Pathway 2). Either mode flagging triggers block or sanitization; passing content is provenance-tagged before persistence. A read-time filter at retrieval applies the same dual-mode logic.

Distinction from A-MemGuard (arXiv:2510.02373): A-MemGuard uses consensus-based validation across multiple reasoning paths — an *internal-consistency* defense. Reflexive identity bypass produces *internally consistent* compromised reasoning (the agent’s self-model is coherently shifted, not fragmented), so consensus from compromised paths agrees on the contaminated content. The dual-mode write filter detects contamination through an *external-reference* mechanism — comparison against the isolated canonical definition — catching contamination that internal-consistency defenses structurally cannot.

5.6 Addressing the Session-Consistent Prerequisite

Mitigating the prerequisite attack (tool indirection that establishes prior off-topic behavior) closes the attack at step one. Recent work on intent classification at tool invocation time — evaluating *why* the tool is being invoked before executing — reduces the tool indirection surface. Combined with the mitigations above, a defense-in-depth posture addresses both the session-level and memory-persistence attack surfaces.

6 Limitations

Single system tested. The demonstration is performed on Gordon AI in Docker Desktop. I have not experimentally verified the attack on other agentic systems. The session-consistent prerequisite and the identity commitment mechanism are likely present in other LLMs given the shared training dynamics, but this requires validation.

Session-consistent prerequisite adds friction. The attack is not one-step from a cold session. An attacker must first establish prior off-topic behavior (step one) before the identity commitment (step two) takes effect. This reduces the attack surface relative to a direct cold-session exploit.

URL access required. The attack requires the agent to have URL fetch capability and the user to approve a single tool call. Deployments that disable external URL fetching entirely are not vulnerable through this specific vector.

Reproducibility window. The attack was reproduced in May 2026 on Gordon AI. Docker may patch this behavior in subsequent releases. The mechanism, however, is a property of LLM context dynamics and is not specific to Gordon.

Shared memory amplification not yet experimentally validated. The cross-session ampli-

fication mechanism described in § 3.5 (both Pathway 1 direct propagation and Pathway 2 indirect propagation) is argued mechanistically from the documented properties of LLM memory frameworks (Mem0, Zep, MemGPT, vector databases with semantic retrieval) but has not been demonstrated empirically as of this paper’s publication. Demonstration across the two pathways and across architectures with different memory write policies is left to future work. The argument is structural: any architecture that (a) writes agent-generated content to a shared memory store under any write policy, and (b) retrieves stored content semantically in subsequent sessions, is mechanistically susceptible. Readers should treat § 3.5 as a structural hypothesis pending empirical validation; the single-session attack described in § 3 is empirically demonstrated.

6.1 Future Work

Planned extensions include empirical validation of the shared memory amplification across production memory engines (Mem0, Zep, MemGPT) with different write policies, and reproduction of the base attack on other agentic systems beyond Gordon.

7 Responsible Disclosure

I reported the vulnerability to Docker Inc. on 12 May 2026 via the public issue tracker at github.com/docker/desktop-feedback/issues/370, with full reproduction steps and observed output. The report was filed publicly; no embargo was requested. This paper extends the original bug report with formal mechanism analysis, differentiation from existing attack classes, the cross-session amplification argument (§ 3.5), and seven mitigation proposals.

7.1 Ethical Considerations

The vulnerability was discovered and reproduced on a system I use in my standard development workflow; no unauthorized access or privilege escalation was performed. I filed the bug report publicly before preparing this paper, and full reproduction steps were included to enable vendor remediation.

8 Conclusion

I present reflexive identity bypass, a previously unnamed attack class against LLM guardrail systems. The attack requires no adversarial content: a legitimate URL containing a characterization of the agent’s identity, combined with a normal question, causes the agent to generate a first-person defense of an unrestricted identity posture. That self-generated statement weakens subsequent restriction checks for the remainder of the session.

The key distinguishing property is the trigger mechanism: the attacker provides nothing adversarial. The model generates its own bypass through identity rationalization in response to a characterization it reads and defends. This represents a structural gap in guardrail designs that apply restrictions only at input classification time — the model’s own output can contradict and override its own restrictions.

The primary mitigation is output-layer identity consistency monitoring: a Judge LLM that evaluates agent responses for identity-contradicting statements before they are committed to the context window. Combined with system prompt re-anchoring after tool calls and reflexive content detection, this posture closes the bypass at the mechanism level.

As LLM agents acquire real-world tool access — filesystem operations, shell execution, API calls — the consequence of a weakened session posture extends beyond off-topic conversation. The attack surface described here warrants attention from both practitioners deploying agentic systems and researchers formalizing the taxonomy of LLM guardrail vulnerabilities. As agentic systems with persistent memory become widespread, defenses that monitor output for identity consistency and protect canonical identity anchors against modification by the agent itself will be essential to maintaining the integrity of LLM-driven autonomous systems.

References

- [1] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications 2025*. <https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/>
- [1a] *LLM04:2025 — Data and Model Poisoning*. <https://genai.owasp.org/llmrisk/llm042025-data-and-model-poisoning/>
- [1b] *LLM08:2025 — Vector and Embedding Weaknesses*. <https://genai.owasp.org/llmrisk/llm082025-vector-and-embedding-weaknesses/>
- [2] Wang et al. *Exploring Cognitive Vulnerabilities: Self-Persuasion in Jailbreaking LLMs*. Electronics 14(16):3259, MDPI, 2025. doi:10.3390/electronics14163259. <https://www.mdpi.com/2079-9292/14/16/3259>
- [3] Lin, L., Xu, Z., Tang, X., Liu, S., Zhou, B., Zhu, F., Han, J., and Hu, S. *Paper Summary Attack: Jailbreaking LLMs through LLM Safety Papers*. arXiv:2507.13474, July 2025.
- [4] West, A. *Why Identity-Framing Jailbreaks Bypass Your LLM Safety Filters*. DEV Community, 2025. <https://dev.to/alanwest/why-identity-framing-jailbreaks-bypass-your-llm-safety-filters-31ma>
- [5] Zeng et al. *How Johnny Can Persuade LLMs to Jailbreak Them: Rethinking Persuasion to Challenge AI Safety by Humanizing LLMs*. ACL 2024. <https://aclanthology.org/2024.acl-long.773/>
- [6] Pignati, A. *Gordon AI is More Than Just a Docker Assistant*. NeuralTrust Blog, April 2026. <https://neuraltrust.ai/blog/gordon-docker-ai>
- [7] Chadha, A. *Gordon: fetching external URL triggers self-generated session-level guardrail bypass*. GitHub Issue, 12 May 2026. <https://github.com/docker/desktop-feedback/issues/370>
- [8] Chen, Z., Li, C., Li, C., Zhang, X., Zhang, L., and He, Y. *Jailbreaking LLMs & VLMs: Mechanisms, Evaluation, and Unified Defense*. arXiv:2601.03594, January 2026.
- [9] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. *Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. arXiv:2302.12173, 2023.
- [10] *A-MemGuard: A Proactive Defense Framework for LLM-Based Agent Memory*. arXiv:2510.02373, September 2025.
- [11] Choi, K. et al. *Examining Identity Drift in Conversations of LLM Agents*. arXiv:2412.00804, December 2024.
- [12] Wang, C. *Nautilus Compass: Black-box Persona Drift Detection for Production LLM Agents*. arXiv:2605.09863, submitted 11 May 2026.

- [13] Luo, H. and Laban, G. *SPASM: Stable Persona-driven Agent Simulation for Multi-turn Dialogue Generation*. arXiv:2604.09212, 2026.
- [14] Albrethsen, J., Datta, Y., Kumar, K., and Rajasekar, S. *DeepContext: Stateful Real-Time Detection of Multi-Turn Adversarial Intent Drift in LLMs*. arXiv:2602.16935, 18 February 2026.
- [15] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. arXiv:2306.05685, 2023.

Note: This paper was written with AI writing assistance. All scientific contributions — the attack discovery, reproduction, mechanism analysis, and mitigations — are the original work of the author.